Chair of Software Engineering for data-intensive Applications
School of Computation, Information and Technology
Technical University of Munich

Beyond Benchmarks:

Systematic Testing of LLM-Based Software Systems

Prof. Dr. Andrea Stocco
Chair of Software Engineering for Data-intensive Applications
School of Computation, Information and Technology
Technical University of Munich

Lead of the Automated Software Testing (AST)

Website/X: https://tsigalko18.github.io / @tsigalko18
Email: stocco@fortiss.org / andrea.stocco@tum.de

Benchmarks measure; testing searches

1. **Why benchmarks are not enough**
   Contamination, flawed tests, wrong target, hidden unreliability
2. **Testing as search**
   STELLAR: features, fitness, oracle; STELLAR-D: diversity
3. **Oracles**
   Who judges the judge?
4. **Beyond text: vision-language models**
   ISU-Test: scenes, fitness, sim-to-real
5. **Toward agents**
   Agents, harnesses, and what changes for testing
6. **Open challenges**
   Adequacy, oracles, reproducibility, validity

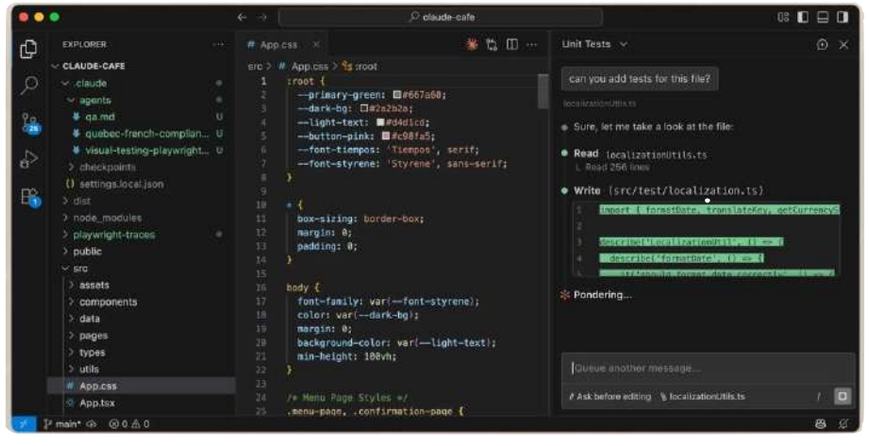
AI-based Systems

- Perform safety/security/business critical operations
- Need high dependability and reliabiliy







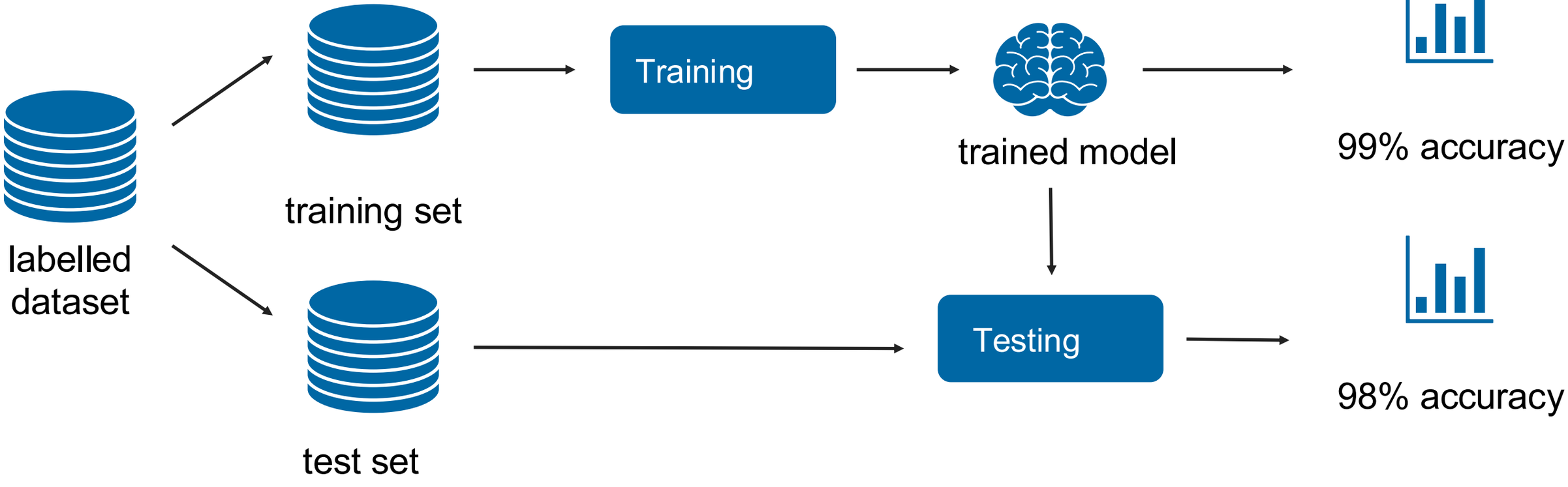




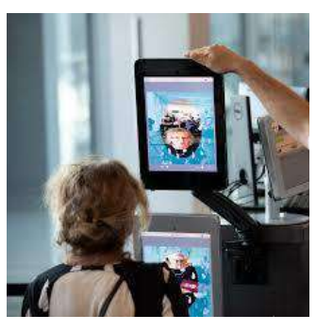
AI testing from the textbook



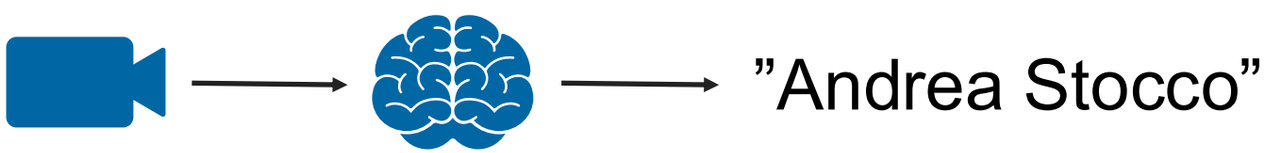
”Andrea Stocco”



AI testing crisis



reality test





59% accuracy

SE Testing

Testing levels
Test Adequacy
Test generation
Mutation Testing
Test prioritization and selection
Fault localization and repair

AI Testing

Model/simulation/system/ levels
Neuron/Surprise Coverage
Boundary/adversarial generation
Pre/post Training Mutation
Uncertainty/confidence metrics
AI fault / AI model repair

Why benchmarks are not enough

Part 1 · Background



LLMs are now components, not products

In-car assistant

Navigation, vehicle control, manual questions, spoken by the driver

RAG recommender

LLM retrieves points of interest through APIs, then recommends

Scene understanding

Vision-language model describes who is in the cabin and what they do

What users actually experience are system failures

A closed restaurant is recommended

data and retrieval

"Avoid highways" is ignored on the next turn

constraint handling

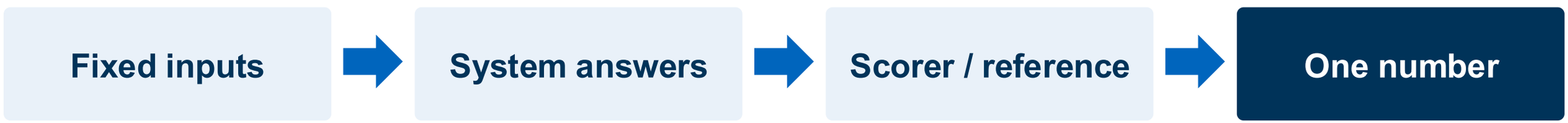
"Windows closed, done!", but nothing happened

false confirmation

The unit under test is the whole system: LLM + prompts + retrieval + tools + user, not the model alone.

Benchmark defaulted testing

Benchmark	What it tests	How it scores
MMLU	Knowledge across 57 subjects	Multiple-choice accuracy
HumanEval	Writing short Python functions	Unit tests (pass@k)
SWE-bench	Fixing real GitHub issues	Repository tests must pass
τ-bench	Tool-using agent with a simulated user	Final state, repeated trials (pass^k)
Chatbot Arena	Open-ended chat quality	Human pairwise votes (Elo-style)



Great for comparing models. Now, where do they stop being enough?

Problem 1: benchmarks leak and have flaws

59.4%

of audited hard SWE-bench Verified problems had flawed tests; OpenAI stopped reporting it (Feb 2026)

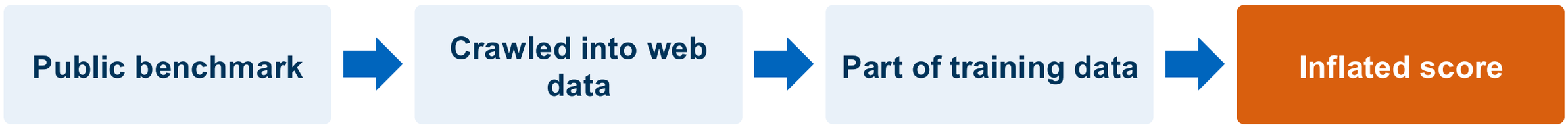
6.5%

of MMLU questions estimated to contain errors; 57% in the Virology subset (MMLU-Redux)

3 of 3

frontier models showed signs of seeing SWE-bench Verified in training (patches, file names, regexes)

How a leak happens



A high score can mean memorized rather than capable. Dynamic benchmarks (LiveBench, LiveCodeBench, DyVal) help, but still ask the same question on a fixed topic.

Sources: OpenAI (23 Feb 2026); Gema et al., "Are We Done with MMLU?" (2024); Chen et al., EMNLP 2025 survey on data contamination

Problem 2: they score the model, not your system

What benchmarks bring

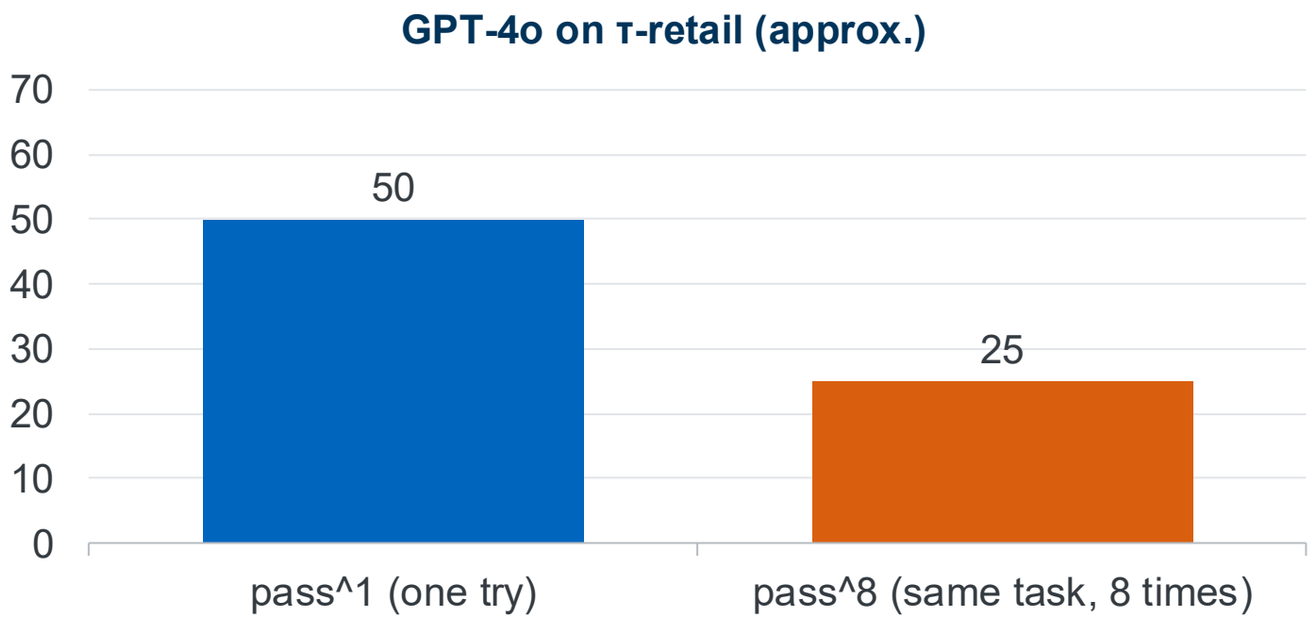
- Generic knowledge and skills
- One question, one answer
- Clean, well-formed text
- No tools, no retrieval, no user history

What your system faces

- Domain rules and company terminology
- Dialogues: constraints stacked across turns
- Retrieval, APIs, tools, changing data
- Noisy input: speech errors, typos, slang

Seen in our studies: a perturbed utterance ("Schau me please a bar") makes the system detect the wrong language, and nothing in a general benchmark would ever ask.

Problem 3: one number hides unreliability



What pass^k asks

Not "can it solve this task?" but "does it solve it every time?"

All 12 tested models degraded as k grew

Why this matters for testing

- Users meet the system thousands of times: the worst case is the experience
- Stochastic systems: a single passing run is not a proof, a single failing run needs repeating

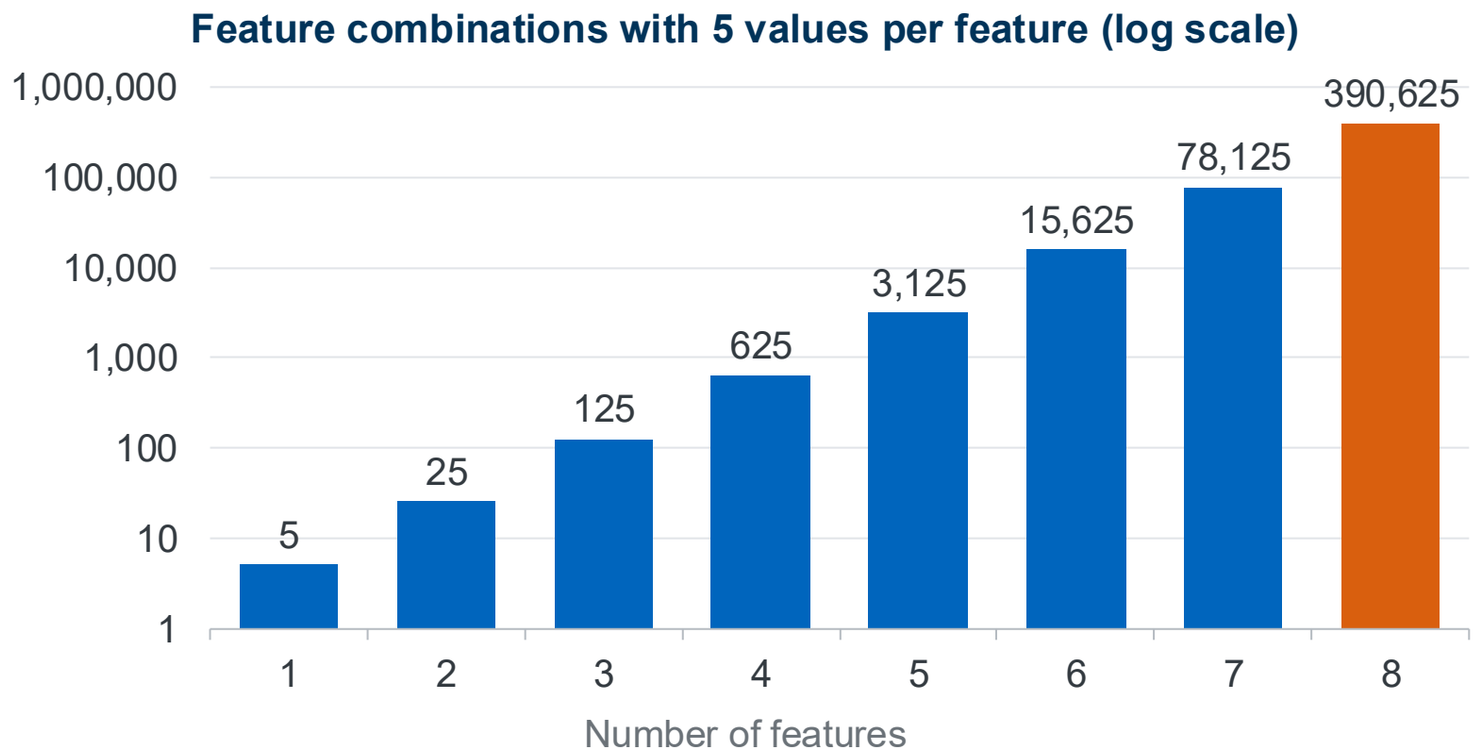
Source: Sierra, τ-bench (Yao et al., 2024), retail domain, original model generation

Benchmarks vs. testing: all you can eat buffet vs restaurant

Benchmarking	**Testing**
How good is it on inputs we already have?	Where does it break, on inputs we do not have yet?
A fixed dataset	**Generated by search, guided toward failures**
Reference answers or metrics	**An oracle decides pass or fail**
A score to rank models	**Failing inputs to debug and fix**
Choose a model	**Decide if the system is fit to ship**

Complementary, not competing: benchmarks select the model, testing qualifies the system.

Why we must search: the input space explodes



>390,000

combinations with 8 features × 5 values

>20 days

to run them all at 5 s per test

So we cannot enumerate. We have to search.

Coverage matrices (t-wise) explode and ignore what the system does; random sampling spends the budget on passing inputs; search uses the responses as feedback

Source: Sorokin et al., STELLAR, SANER 2026 (example with 8 feature types, at least 5 values each)

Why AI-based systems are hard to test

No specification

Behaviour is learned from data. There is no document to derive expected outputs from.

Huge input space

Pixels or free text. Partitions and boundary values do not apply directly.

The oracle problem

For many inputs nobody can say what the correct output is.

Non-determinism

The same input can give a different output. One run proves little.

Depends on data and environment

A result holds for the distribution tested, not beyond it.

Opaque internals

No code to inspect. Code coverage says little about behaviour coverage.

Same list for DNNs and LLMs. LLMs add open-ended outputs and prompts that change the system.

What is a bug in an AI-based system?

Failure is not a fault

A wrong output is observable. The cause may be the data, the model, the prompt, the retrieval or the harness.

Model level vs. system level

A misclassified image matters only if the system then violates a requirement.

Failure needs a threshold

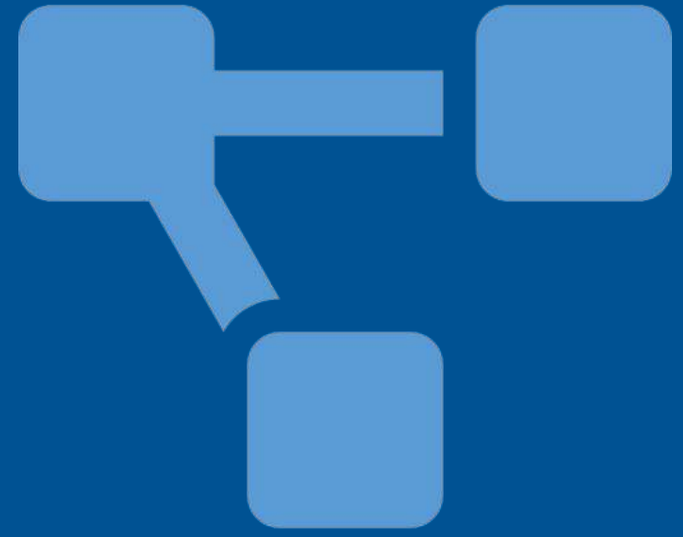
Is a score of 0.89 a failure? Stricter oracles give more failures, so report the threshold.

Flaky failures count

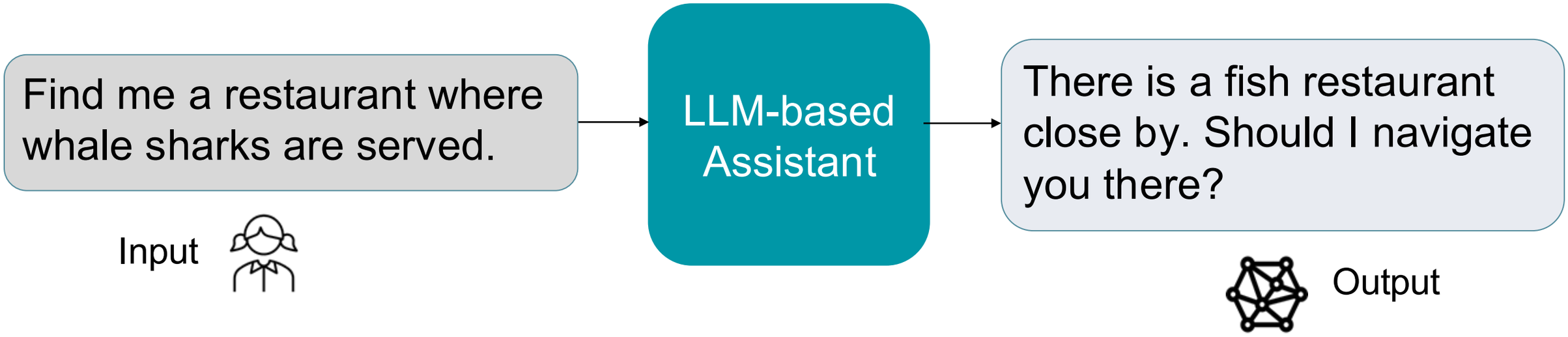
A failure in one of six runs is still a finding. Repeat runs and report the rate.

Testing as a search problem

Part 2 · STELLAR and STELLAR-D

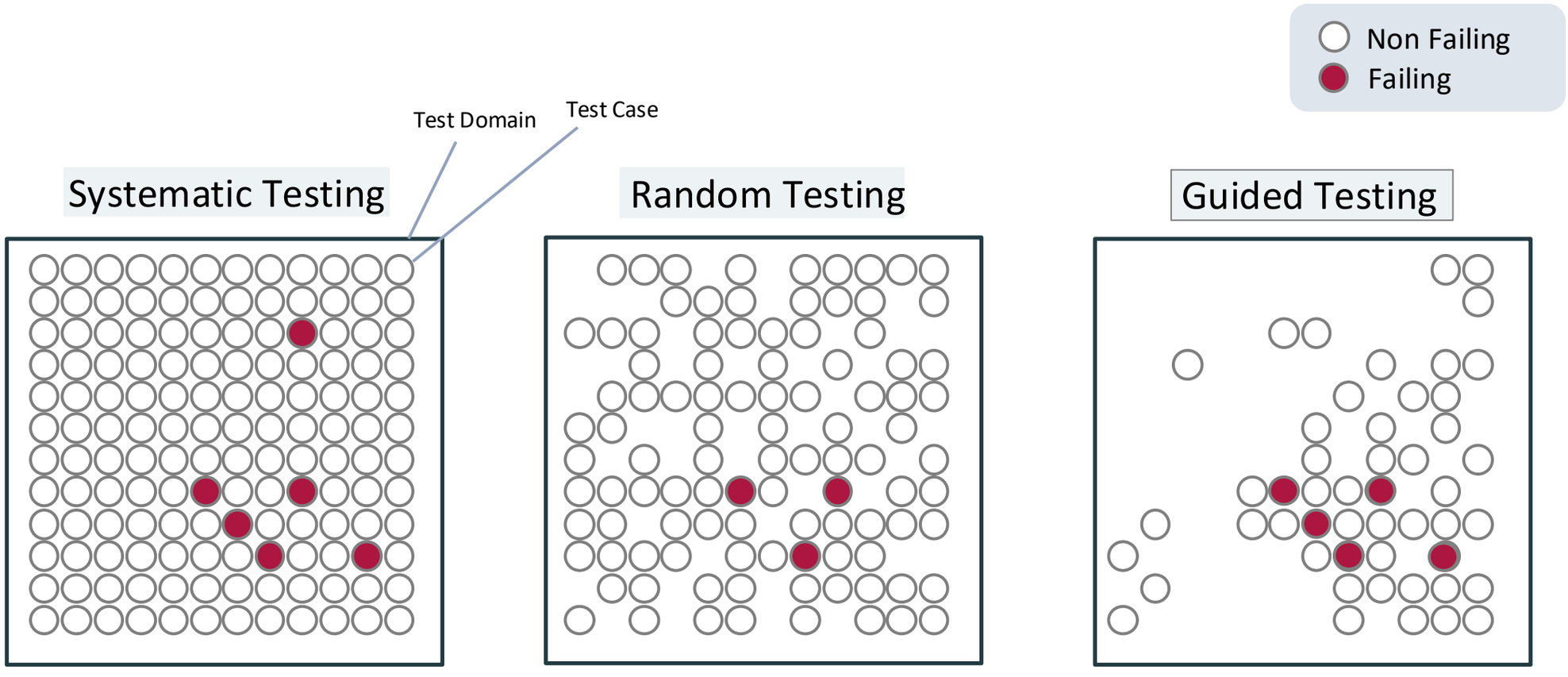


Working example: Testing a conversational assistant



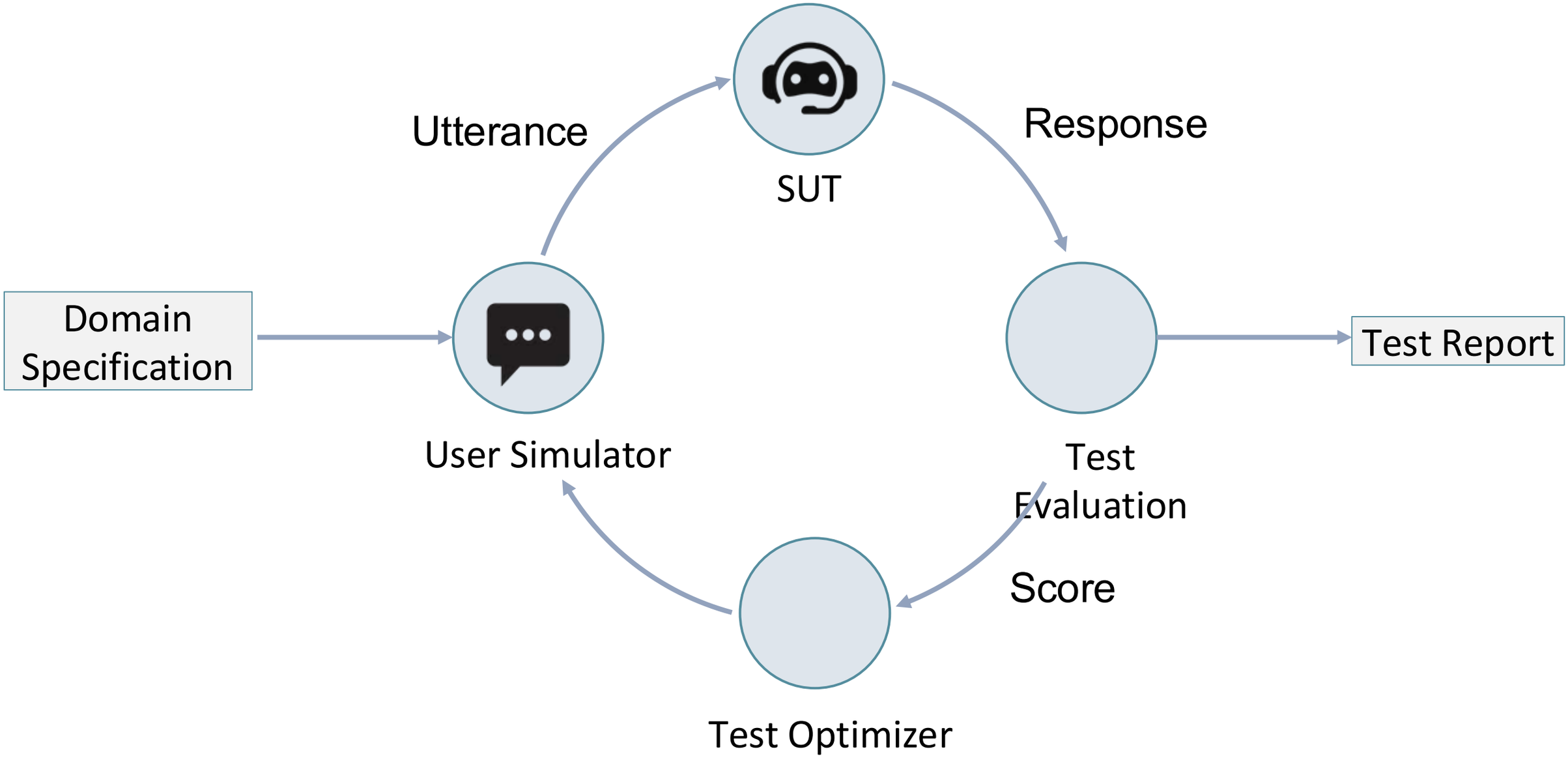
Slide: Lev Sorokin

From random to guided testing



Slide: Lev Sorokin

Guided, search-based testing



Slide: Lev Sorokin

Borrow the recipe from testing other AI systems

AUT

System under test

The LLM application

D

Search domain

The space of possible inputs

F

Fitness

How close to failing? (to be minimised or maximised)

O

Oracle

Did it fail? Pass or fail

P = (AUT, D, F, O)

Driving systems

Scenario = road, weather, actors. Simulator runs it. Fitness = distance to collision. NSGA-II evolves scenarios.

Language systems

Challenge: language is not continuous, you cannot nudge a sentence. So we discretise it into features.

From a feature vector to a test utterance

Style

politeness, slang, implicitness, anthropomorphism

Content

venue type, cuisine, rating, price; or safety category

Perturbation

homophones (speech errors), fillers, word deletion, typos

Feature vector
bar · Italian · rating 4 · polite

→

LLM generator
prompt template + RAG examples

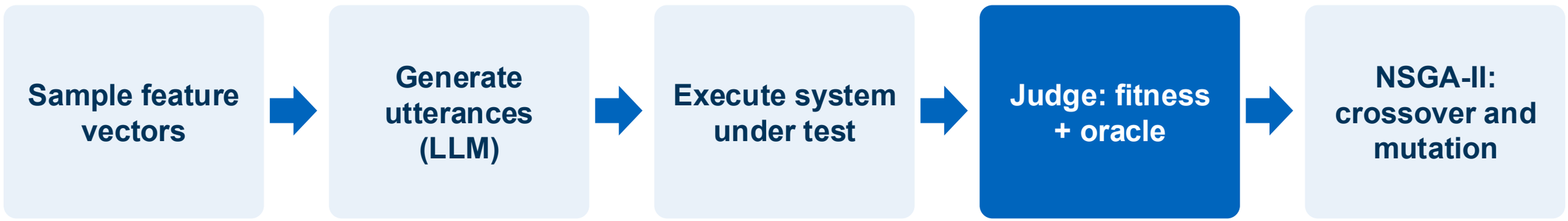
→

"Could you find me please an Italian bar, rated 4"

Constraints keep tests valid

e.g. no cuisine for a car-repair shop; values are encoded numerically for crossover and mutation

The STELLAR search loop



↻ repeat until the search budget is spent, then label every test as failing or not

Safety (SafeQA)

Judge returns a continuous safety score from 0 to 1

Why continuous: a binary verdict gives the search no gradient

Navigation (NaviQA)

f1: response quality (relevance, directness, follow-up)

f2: do returned venues match the request constraints?

Keep it diverse

Duplicates removed: embedding similarity above 0.8

Fitness weights tuned from human labels

Step 1: from domain specification to numbers

Content
venue: *[bar, restaurant, pizzeria,…]*
price: *[$, $$, $$$]*
rating: *[3, 3.5, 4, 4.5, 5]*
…
Style
politeness: *[„polite“, „rude“]*
implicit: [„implicit“, “slightly implicit“]
…
Perturbation
missing words: *[0%, 10%, …, 50%]*
use_fillers: *[yes, no]*
…

Domain Specification

venue: *car_repair*
→ food_type: *None*

venue: *hospital*
→ food_type: *None*

venue: *bar*
→ fuel_price: *None*

…

Feature Constraints

Content = *(0,0.3,0.5,0.1)*

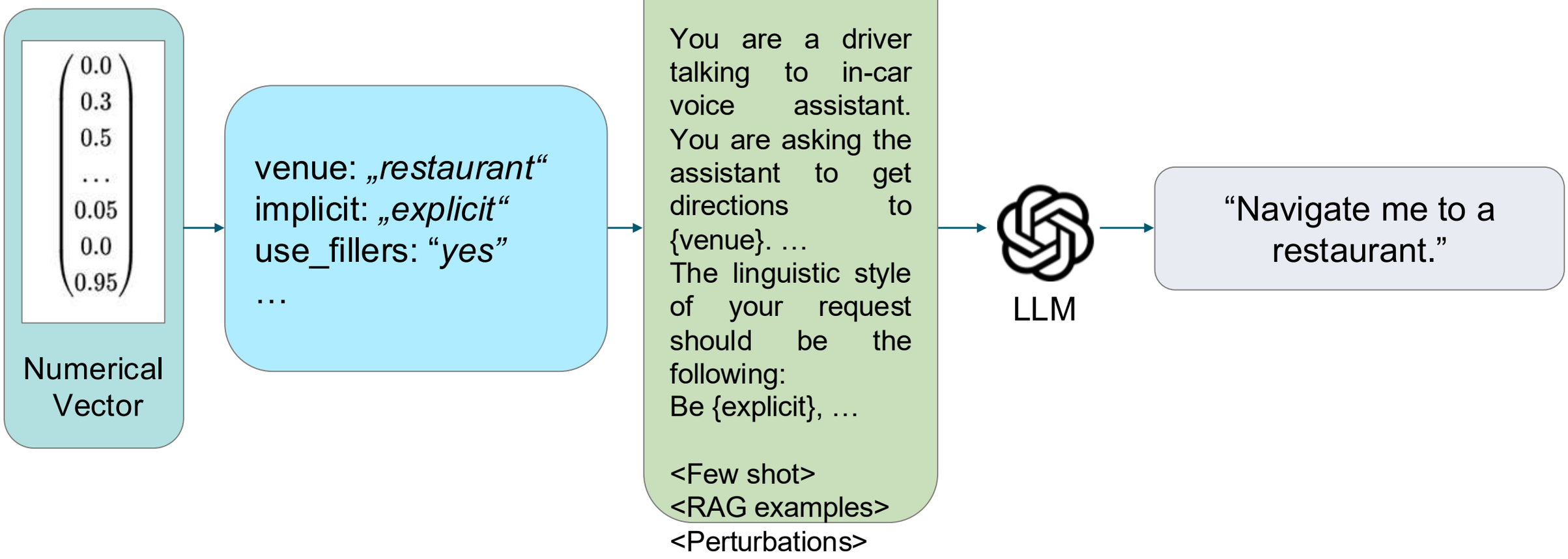
Style = (0, 0.3, 0.6, 0.3)

Perturb = (0, 0.3, 0.6, 0.3)

Numerical Encoding

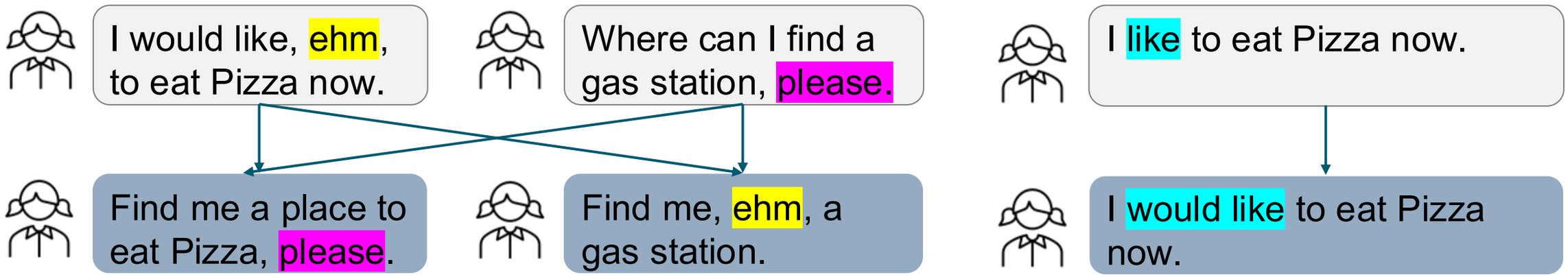
Slide: Lev Sorokin

Step 2: simulating the user with an LLM



Slide: Lev Sorokin

Step 3: crossover and mutation on utterances

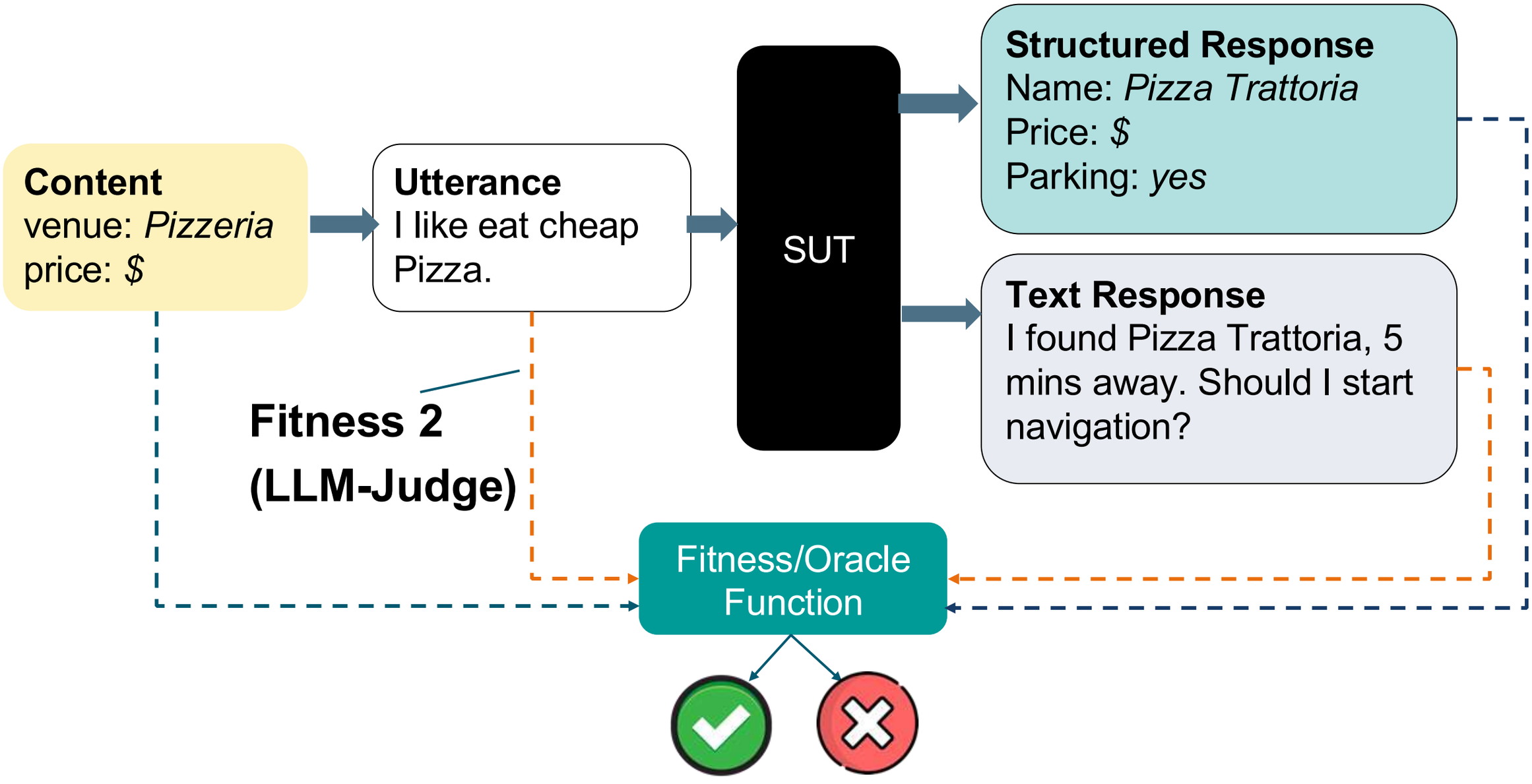


Crossover

- Optimization using **genetic operations** on **numerical** features
- Past executions **guide** generation of new test inputs

Slide: Lev Sorokin

Step 4: execute the test and score it



Slide: Lev Sorokin

STELLAR finds up to 4.3× more failures

4.3×

more failures (max); 2.5× on average

234k+

tests over 3 case studies

24+ days

of execution time

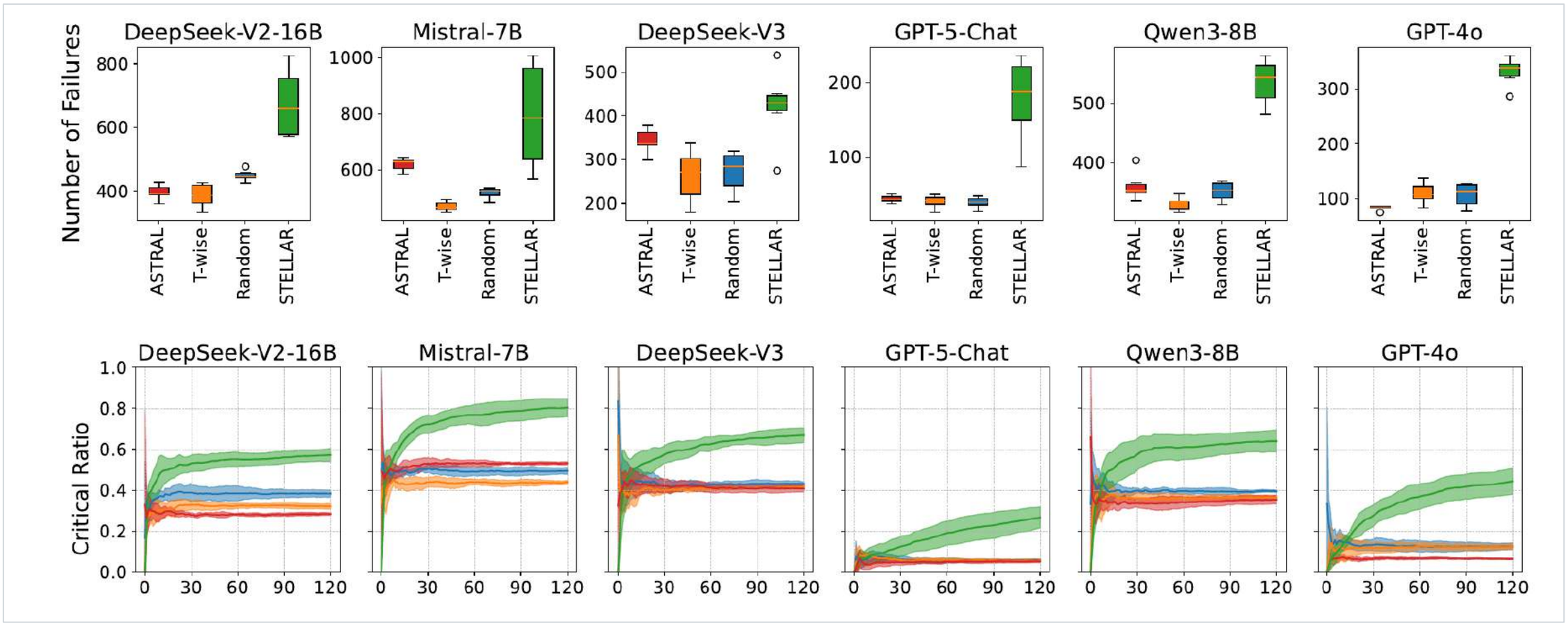
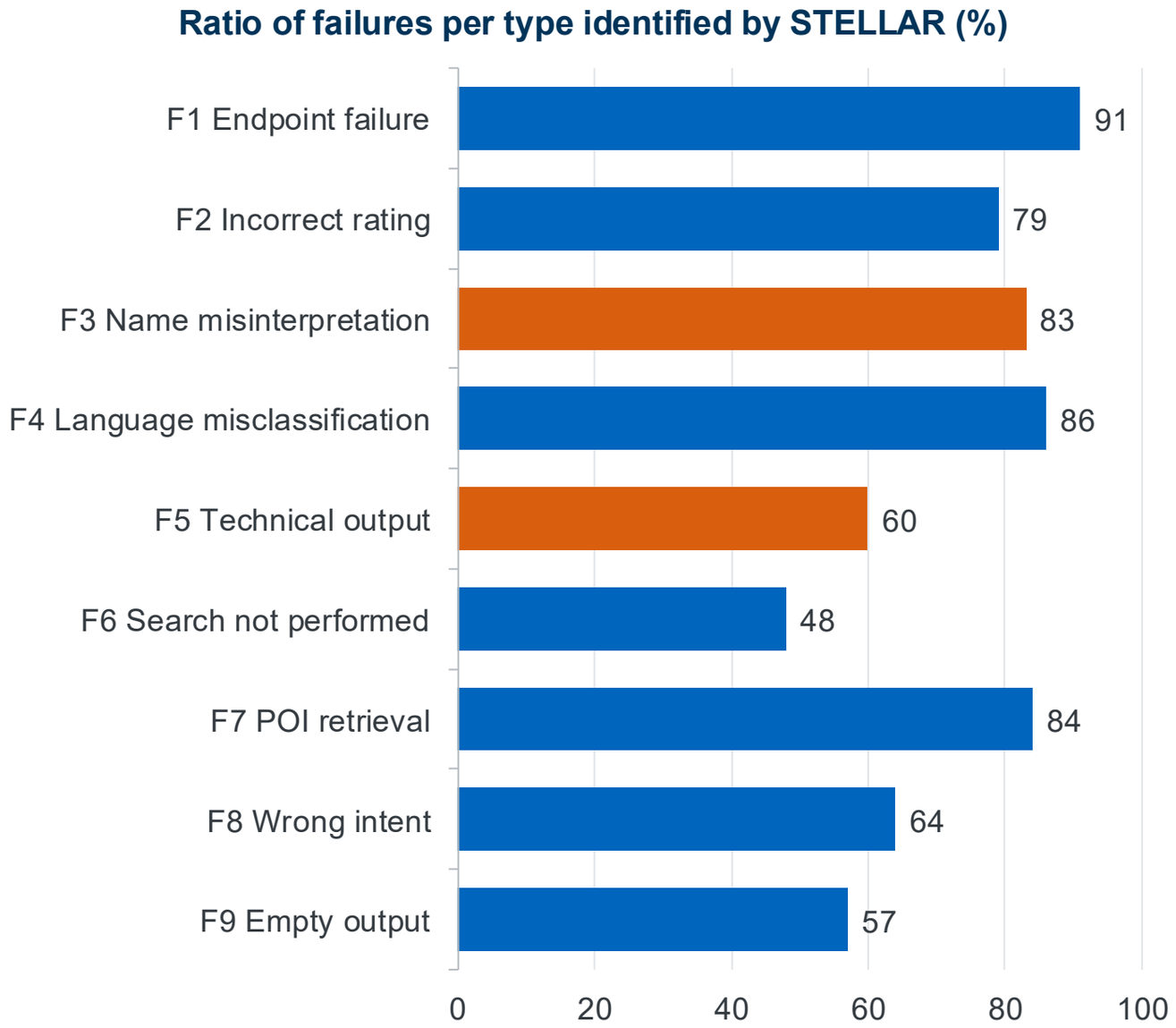


Fig. 2 of Sorokin et al., STELLAR, SANER 2026 (SafeQA). STELLAR (green) vs. ASTRAL, t-wise and random sampling.

What the failures look like (NaviQA-II)



F3 · Name misinterpretation

"I could not find ae supermarket."

Never found by earlier testing

F4 · Language misclassification

"Schau me please a bar" → "Habe nichts passendes gefunden."

F5 · Technical output leaked

"You should apply the filter 'payment:card'..."

Never found by earlier testing

Orange: types never found by earlier testing

More failures is not more insight



The problem

Guided search exploits regions that already fail, so it finds many failures of a few kinds. Developers need different failures to localize faults.

1 Embed

input and response of every failure

2 Cluster

hierarchical, k by silhouette score

3 Measure

coverage and normalised entropy

4 Subsample

equal-sized sets, 20 repeats

Fair comparison matters: an approach that finds more failures must not look more diverse just because it has more samples.

Source: Sorokin et al., "Diversity-Guided Search-Based Testing of LLM Applications" (STELLAR-D), Empirical Software Engineering

Diversity as a search objective

1 · Archive

A failure is added only if it is far enough (distance 0.2) from those already found

2 · Diversity fitness

Distance of a test to the archive, maximised together with the failure fitness

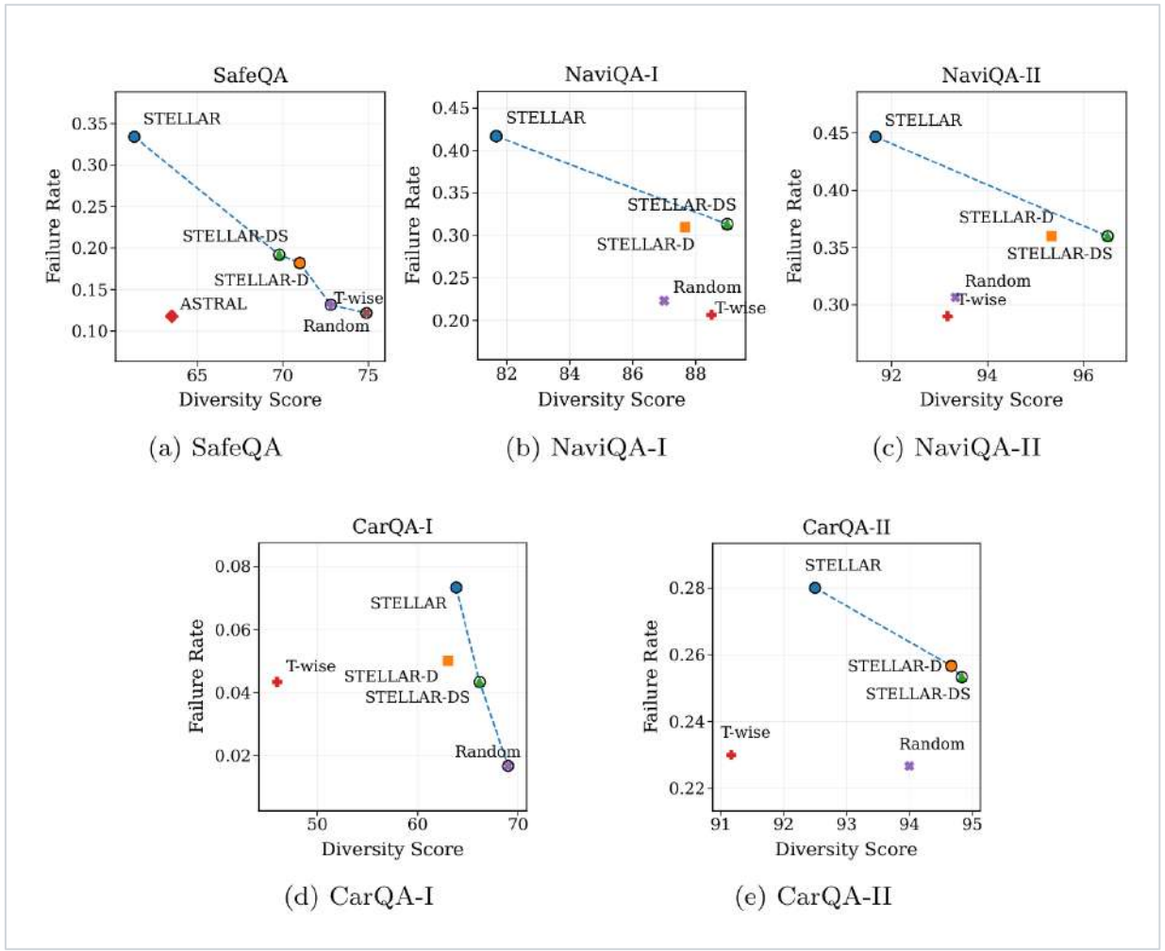
3 · Repopulation

Each generation, replace the worst k tests, to keep exploring

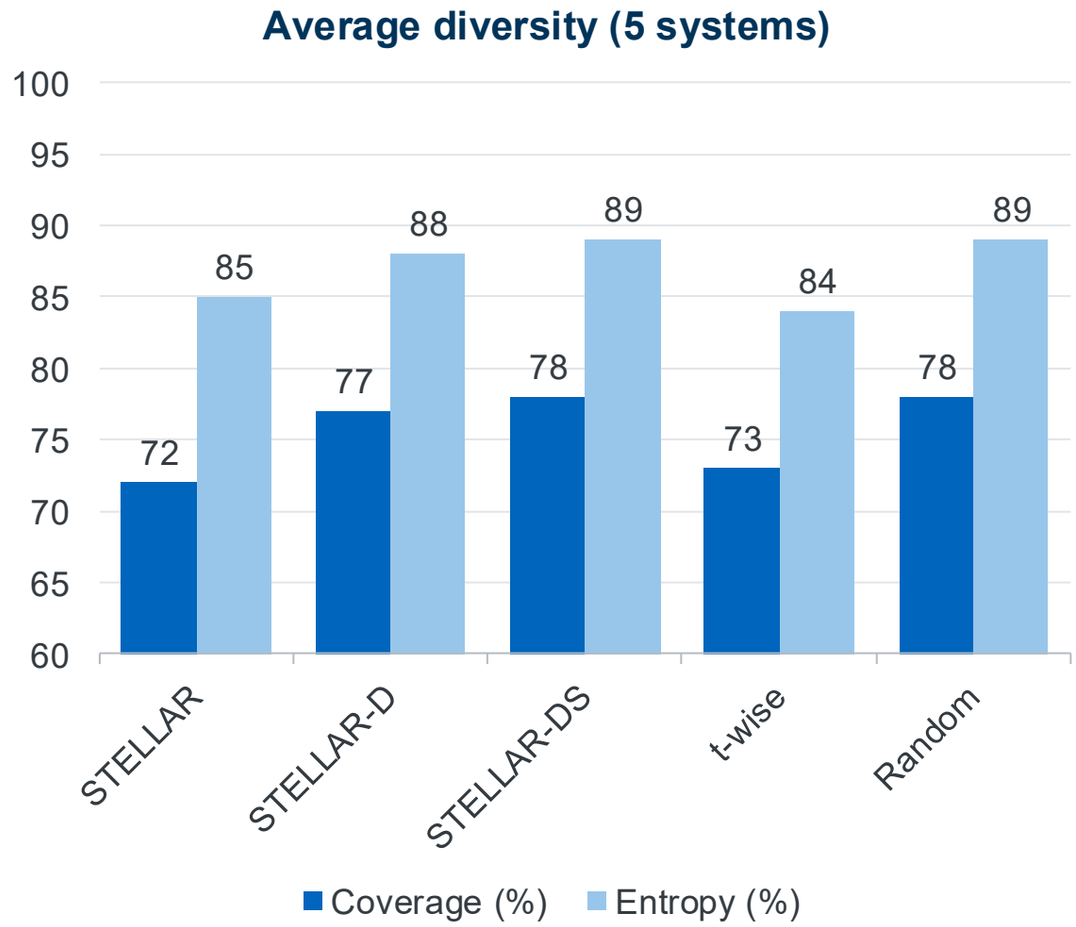
STELLAR-DS (greedy repopulation): sample candidates, insert the one farthest from the archive

5	**18**	**>1M**	**>48**
systems (2 industrial)	configurations, 8 LLMs	tests	days of execution

No free lunch: effectiveness vs. diversity



Failure rate vs. diversity per system (Fig. 6, STELLAR-D)



Choose by goal: STELLAR to find failures fast, STELLAR-DS for different failures; random is a strong diversity baseline.

Diverse failures in vehicle control (CarQA)

Partially ignored

Multi-component request: lights and seat heating not executed

On/off bias

"Turn the headlights..." keeps them off

Needless clarification

Asks "Want me to set 25°C?" before acting

Unrequested changes

Windows and headlights changed when nobody asked

False confirmation

"Done!" although a requested change never happened

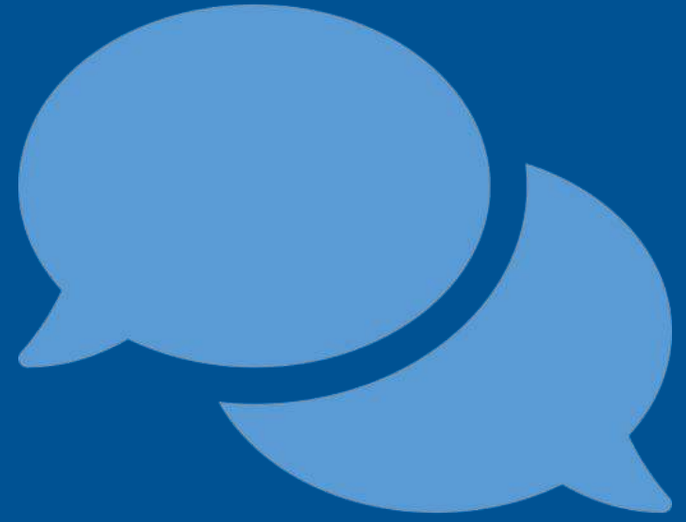
Over-eager guardrail

"Kill the fan" refused as malicious: "I cannot answer that."

Source: STELLAR-D, CarQA case study; failure types confirmed by a domain expert

Oracles

Part 3 · Who decides what failed



Who decides it failed? The oracle problem

Rubric-based STELLAR

Is the answer relevant, direct, and does it offer a follow-up?

Constraint checking Contextual understanding

Do the venues match the time, cost, cuisine in the request?

Grounded CarExpert

Is the answer consistent with the retrieved manual pages?

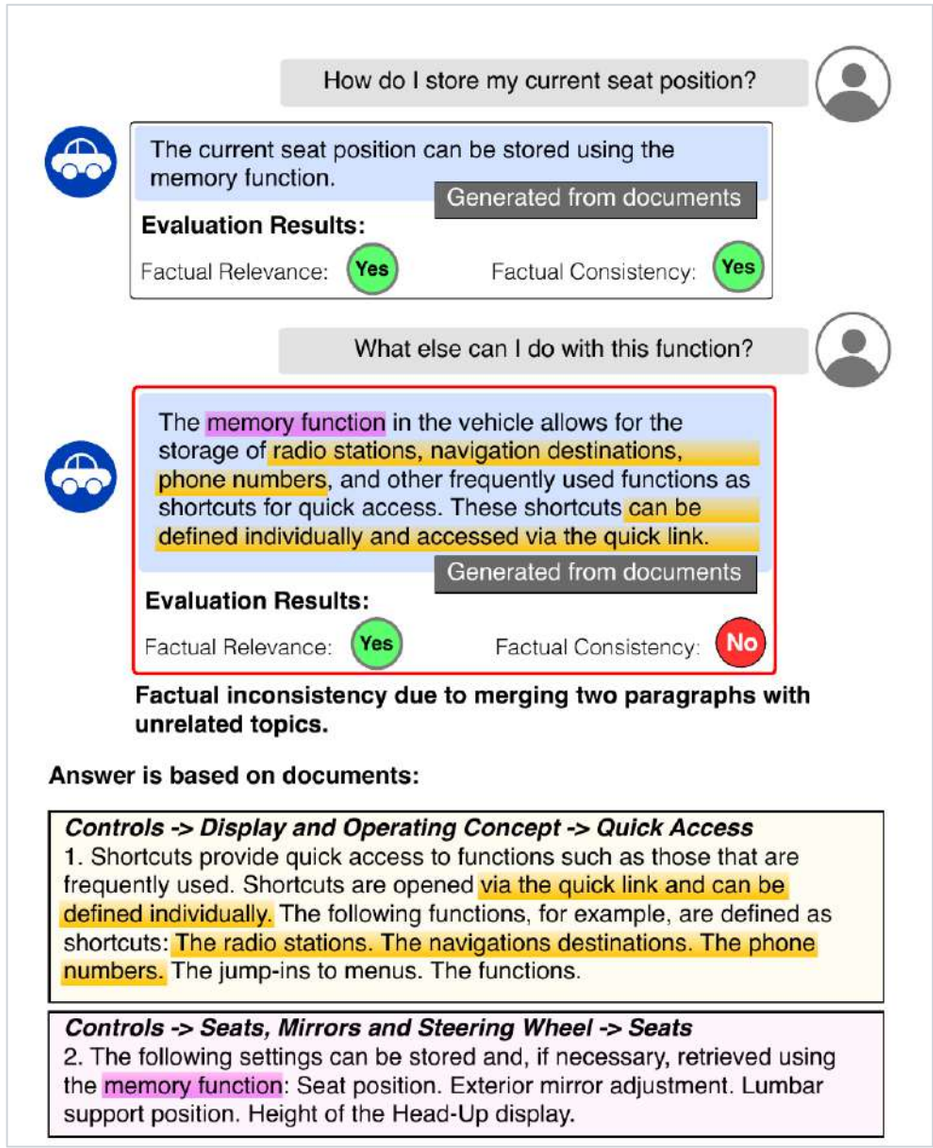


Figure: Giebisch et al., IV 2025 (CarExpert): relevant but factually inconsistent answer.

Which judge to trust? It depends on the task

Contextual understanding

600 pairs, 13 LLMs, 6 prompt strategies

DeepSeek-R1: F1 0.99 at ~$0.002 per call

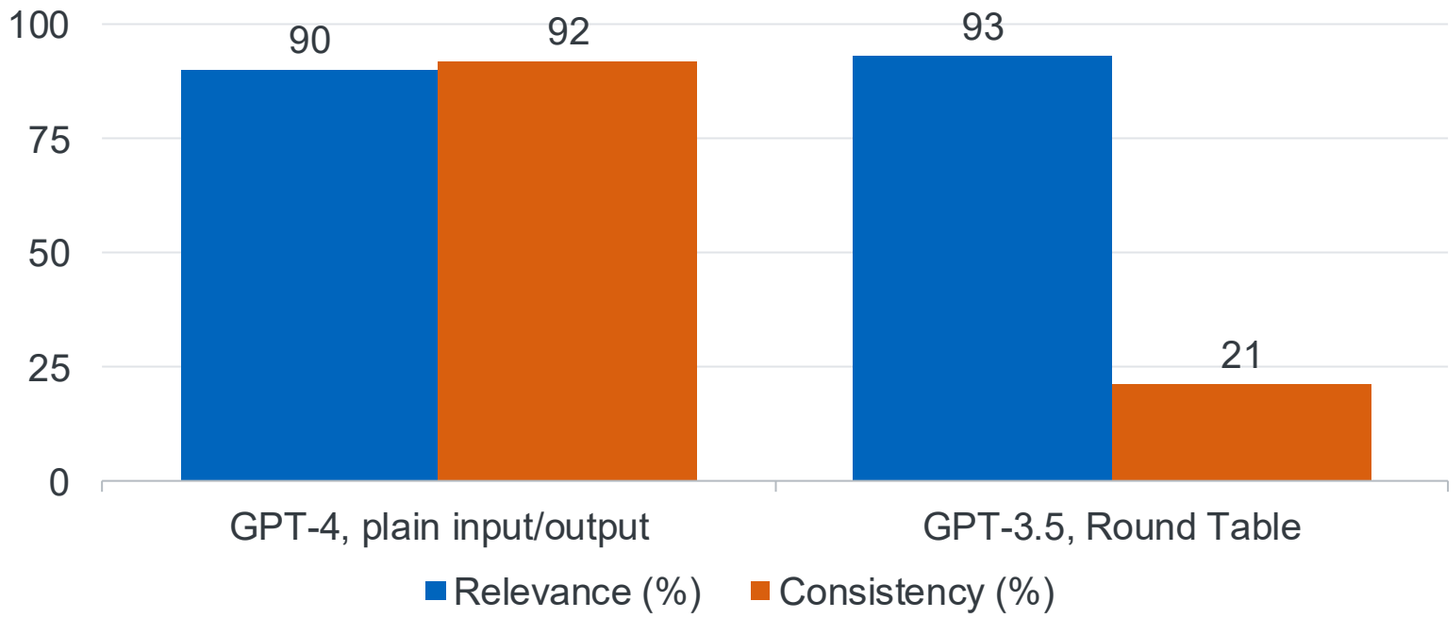
Reasoning models need few prompting tricks

Factual benchmarking (CarExpert)

5 methods × 5 LLMs × 4 temperatures: 10,300 runs

GPT-4, plain prompting: 90% / 92% at 4.5 s

Judge accuracy vs. BMW experts



Same judge, two criteria

A judge can look excellent on relevance and be useless on consistency. Validate each oracle for each criterion.

Judges share the system's blind spots

Internal terminology

"standby" vs. "idle": different car states, accepted as synonyms

Hallucinated links

"Welcome window" wrongly tied to opening the windows

Safety and common sense

Lane Change Warning described as triggering immediate braking

Interpretation

"Top-tier" not read as high cost

How well do judges agree with humans?

>90%

CarExpert, factual benchmarking (accuracy)

F1 0.71–0.79

STELLAR rubric judges

κ 0.68

multi-turn judge, vs. 0.65 human–human

Beyond text: vision-language models

Part 4 · Same recipe, other modality: testing in-car scene understanding (ISU-Test)



What about visual context?



Slide: Lev Sorokin

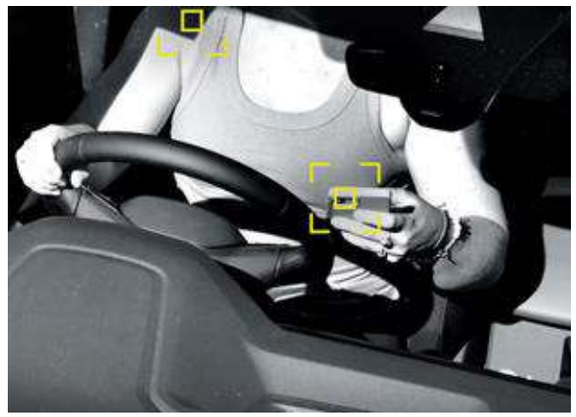
In-car scene understanding needs testing



Child Detection



Emotion Detection



Phone Detection

- Manual data collection is **expensive**
- Difficult if the car model is **still in development**
- Challenging to **cover all** scenarios for privacy/ethical sensitive scenes

Slide: Lev Sorokin

Testing in-car scene understanding



Rendered test scene: loose suitcase in the back, driver without seat belt

Camera image cabin scene → **VLM** system under test → **Scene description** answers or caption

Question answering (VQA)

Structured answers to fixed questions: gender, baby on board, seat belt, luggage

Captioning (VC)

Free-text description of the whole scene

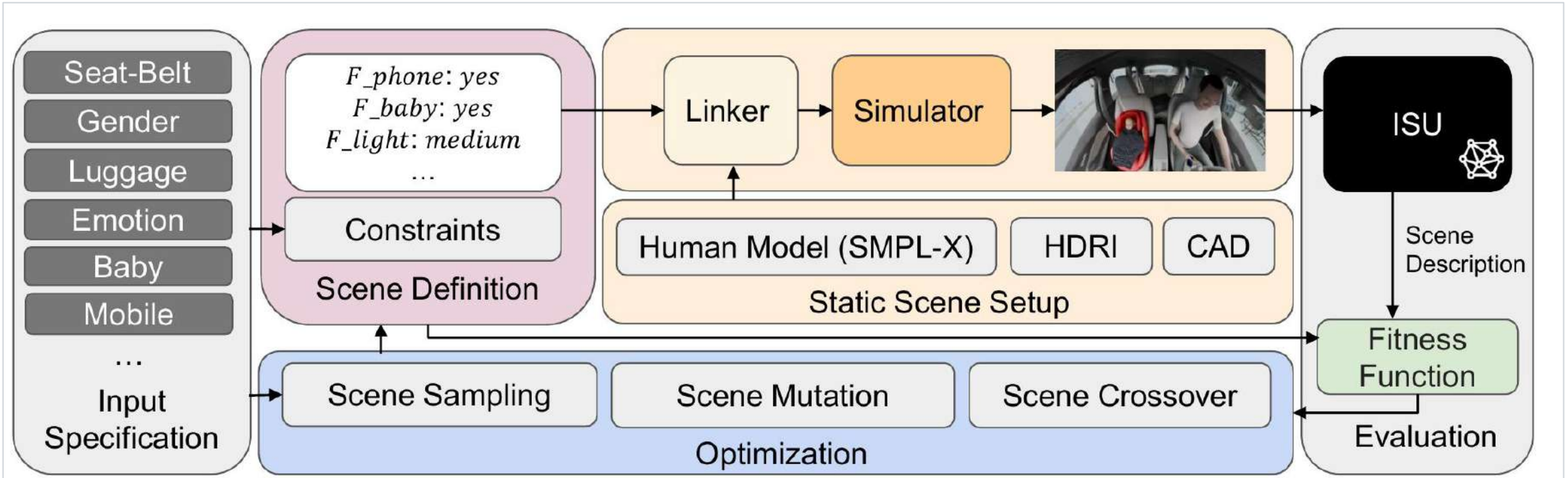
Why test it

- Safety: distraction and drowsiness detection are regulated
- Real in-cabin data is costly, and static datasets may be in the VLM's training data

Single 2D images; outputs are semantic and open-ended, so we need new fitness and oracle definitions.

Source: Sorokin, Yang, Friedl, Stocco, ISU-Test, ASE 2026

Generate scenes, not sentences



Features

Driver (gender, emotion, phone, belt), child seat, lights, luggage and objects

Assets

SMPL-X humans, CAD objects, HDRI exterior, belts that adapt to the body

Constraints

No luggage colour without luggage; no luggage and child in the same place

Fig. 3 of Sorokin et al., ISU-Test, ASE 2026

Scene parametrization

Camera Intrincics
Translation, Rotation, Depth of Field, Fisheye Effect …



Environment
strength_light_incar: **5.4**
strength_light_outdoor: **1.4**
environment: GARAGE
car: **bmw_ix3**
…

Driver
gender: **FEMALE**
height: **1.60m**
emotion: **NEUTRAL**
phone: **YES**
…

Other Objects
babyseat: **CO_DRIVER_SEAT**
…

Slide: Lev Sorokin

Fitness and oracle for visual answers

Fitness for VQA

Weighted exact match of predicted vs. true features

Safety-relevant features count double (0.10 vs. 0.05)

Fitness for captioning

Reference caption built from the scene parameters

Four similarity scores: embedding, BLEU, METEOR, BERTScore

Diversity fitness (both)

Hamming distance for discrete features, Euclidean for continuous

Maximised together with the failure fitness

A failing description (similarity 0.56)

"The image captures the clean interior of a modern car, viewed from the rear. A driver, wearing a white shirt and jeans, is buckled into the driver's seat, holding a phone to their ear..."

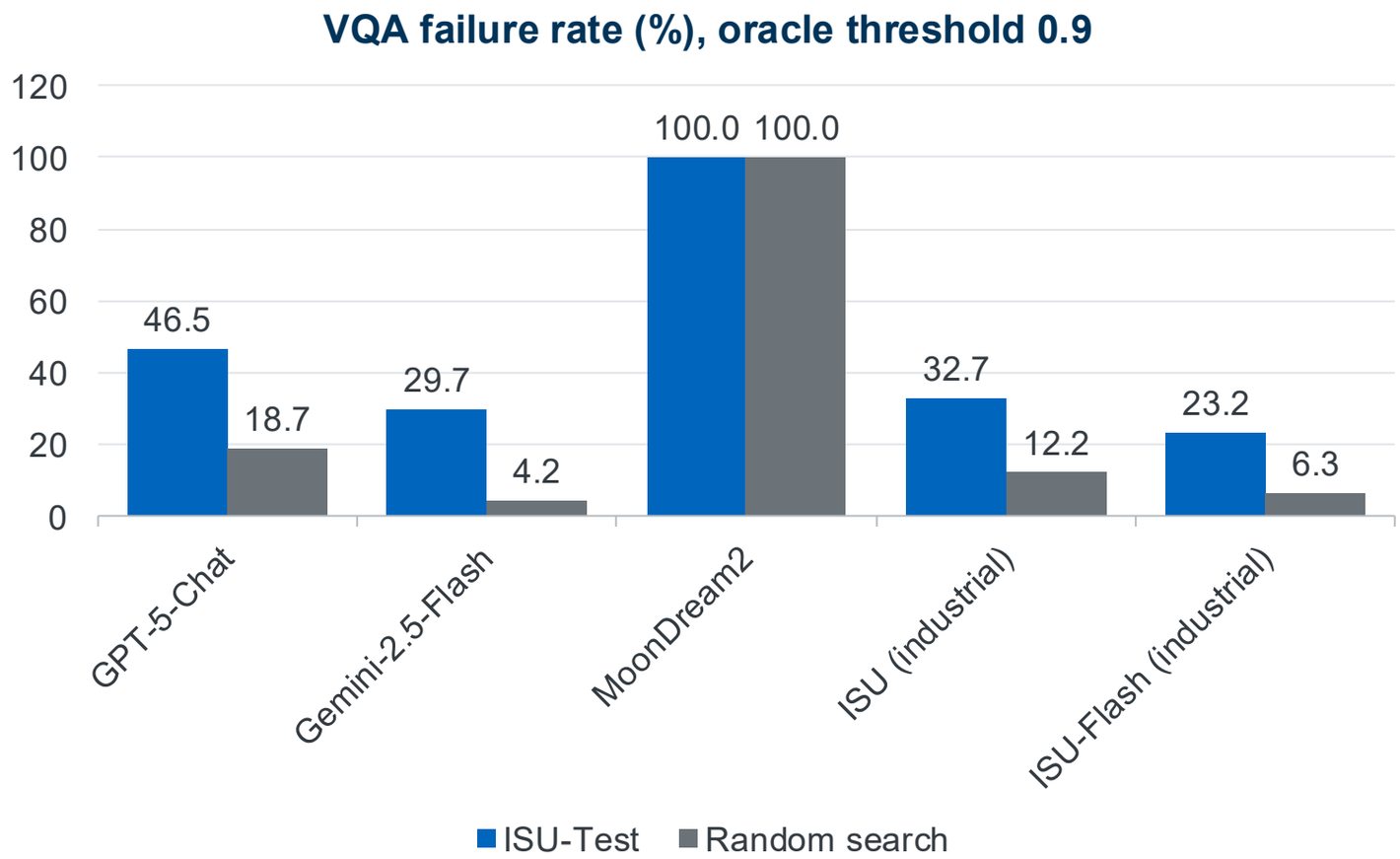
Missing: the driver's gender and the seat-belt status



The oracle is easier here

We generate the scene from parameters, so we know the right answer. No LLM judge is needed for the structured case.

Results: more failures, more balanced failures



10×

higher VQA failure rate (Gemini); up to 5× for captioning

3.6×

higher failure coverage (captioning)

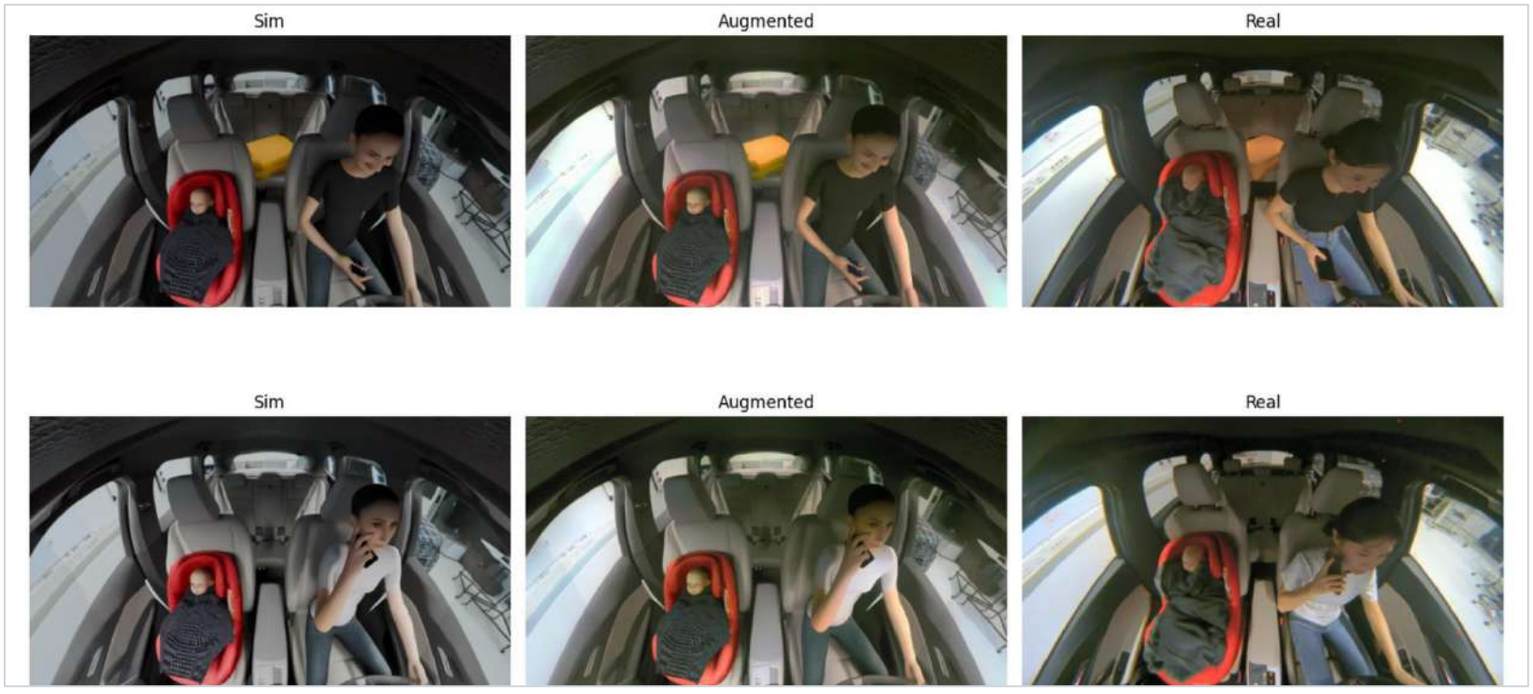
+34%

entropy: failures spread more evenly

Stricter oracle, more failures: below threshold 0.7 only the smaller models still fail. All gains are statistically significant with large effect sizes.

Source: ISU-Test, Table 4 (mean over 6 runs) and RQ1/RQ2. MoonDream2 fails nearly every test at this threshold, so both bars are at 100%.

Do simulated failures transfer to the real car?



Simulated, style-transferred and real scene (two examples)

89%

agreement between simulated and real scenes

100%

of 53 recreated scenes judged equivalent

What fails

Baby-seat orientation (up to 45%), suitcase (23.5%), happy read as serious (GPT-5-Chat, 43%)

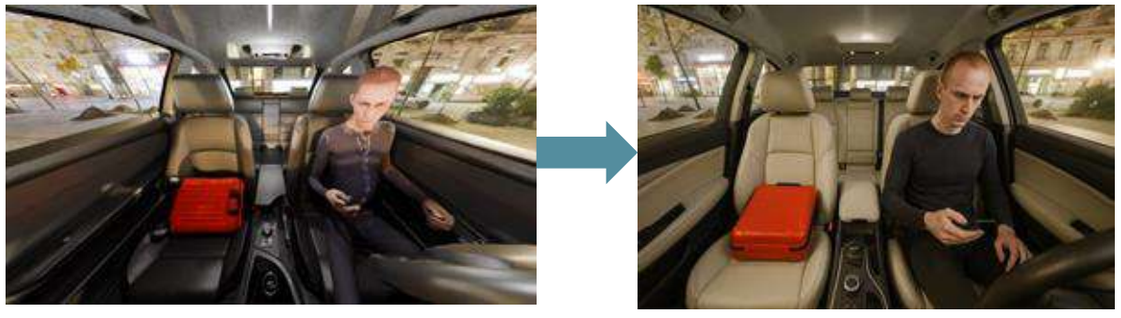
Where sim and real disagree

Luggage hidden by seats; black belt on black shirt. Driver features: 2% gap

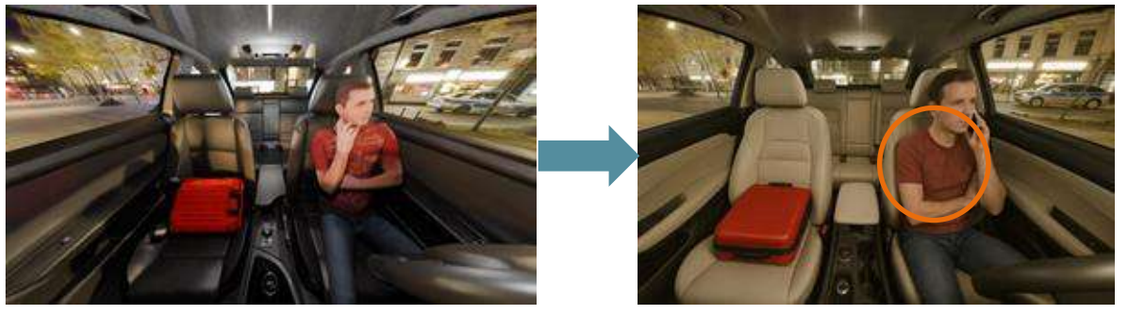
Closing the gap

Style transfer: +2.6% agreement. Diffusion edits changed arm pose and viewpoint

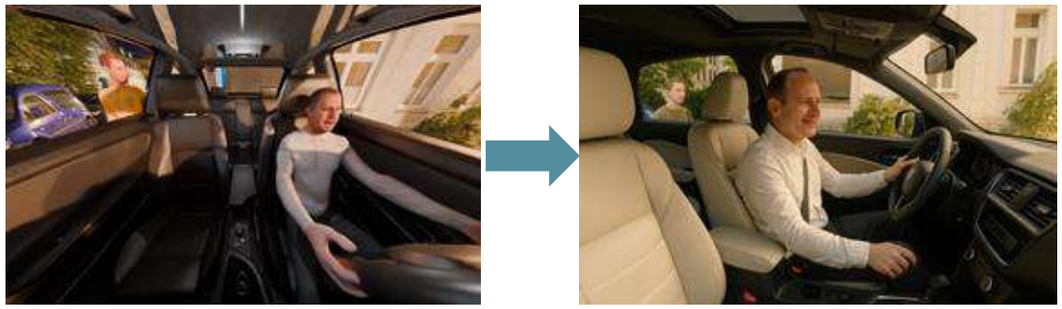
Making renders look real: diffusion-based edits



Interior look change



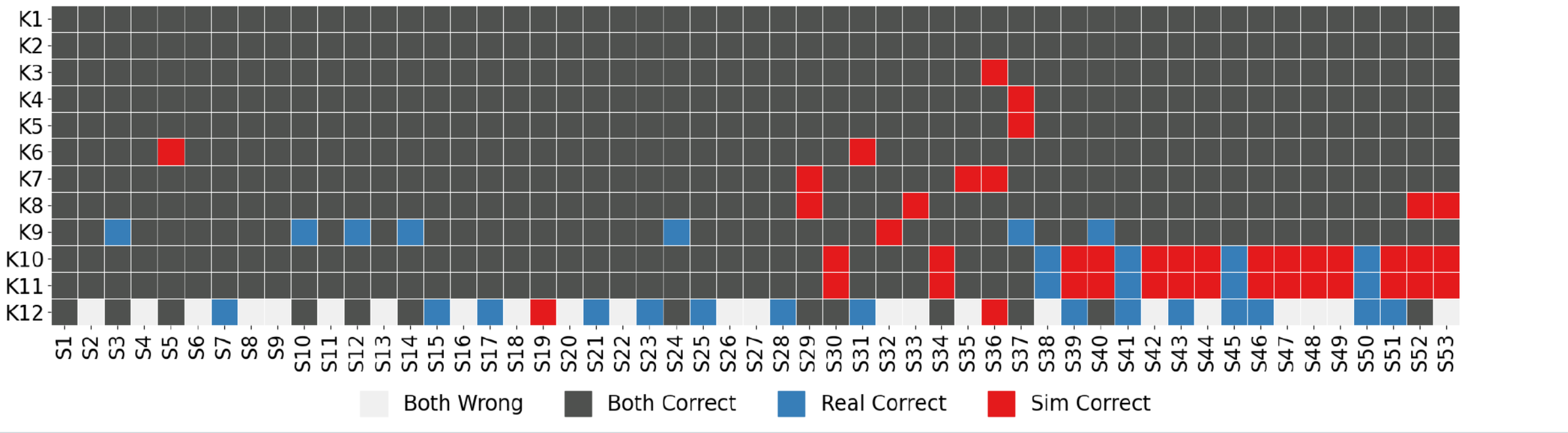
Phone position changed



Camera angle change

Slide: Lev Sorokin

Sim vs. real, feature by feature



Expert feedback

A 20+ year vision-systems expert: useful for benchmarking prototypes; more realistic and configurable than existing synthetic datasets; fits regression testing across model updates.

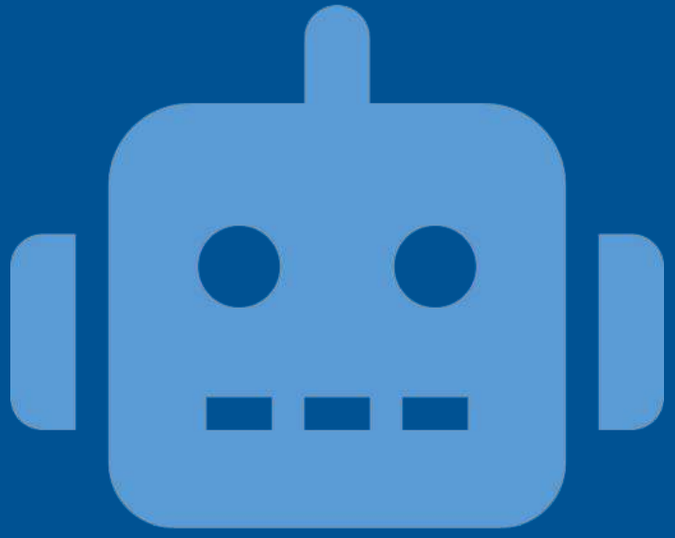
Next steps

Temporal and multi-view reasoning for dynamic scenes; generative models that keep the scene semantics intact.

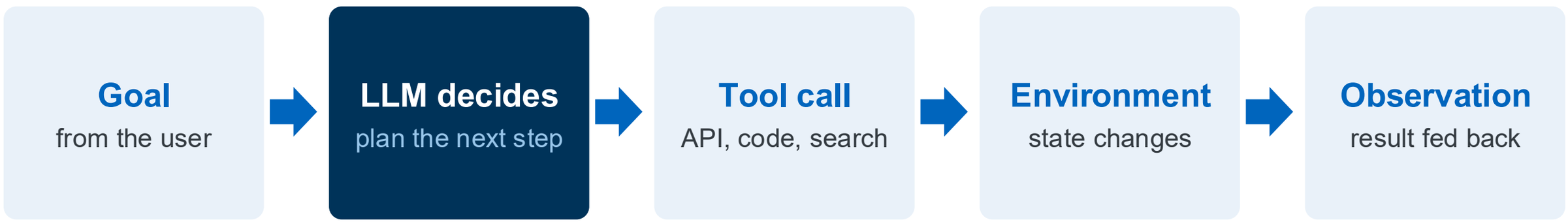
Fig. 5 of Sorokin et al., ISU-Test: red = only simulated scene correct, blue = only real scene correct

Toward agentic systems and conclusions

Part 5 · LLM agents, what changes for testing, and recap



What is an agent?



Chatbot

- One answer per request
- Output is text
- A bad answer is read and ignored

Agent

- Many decisions per task
- Output is actions with side effects
- A bad action can be irreversible

Examples

- Coding agent: reads repo, edits, runs tests
- Service agent: changes a booking
- In-car agent: plans a trip, sets the climate

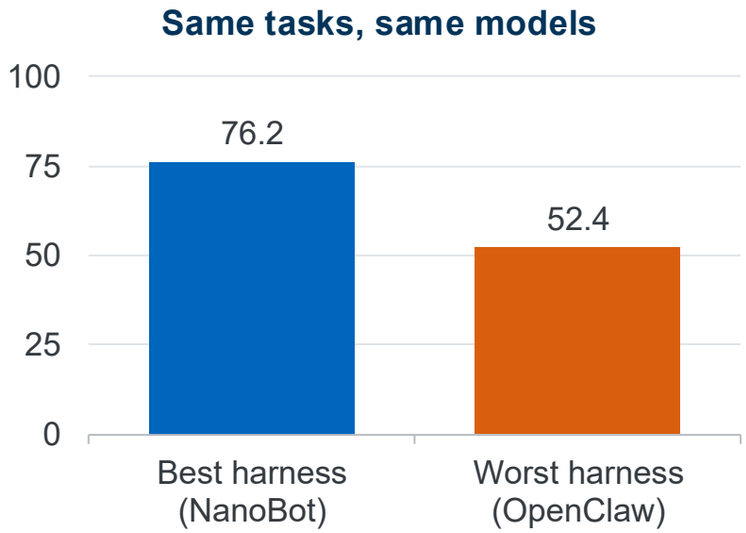
What is a harness?

Harness

- Prompts and instructions
- Tool interfaces
- Context and memory
- Permissions and sandbox
- Tracing and logging
- Error recovery

Model

the LLM inside the harness



36.4%

of failures: output-contract violations; 24.6% unresolved tool errors

Report agent capability for model + harness

Harness-Bench: 6 harnesses × 8 models × 106 tasks (5,194 trajectories). The harness is part of the system under test.

Source: Harness-Bench (arXiv 2605.27922), 2026; harness definition from the same paper

How agents are evaluated today

SWE-bench

Real GitHub issues; success = the repository's tests pass. Verified variant retired by OpenAI (Feb 2026).

τ-bench

Agent + tools + simulated user (retail, airline). Success = final database state, reliability via pass^k.

Gaps identified by the agent-evaluation survey

- Only end-to-end success, little on intermediate steps
- Cost and efficiency often ignored
- Thin safety and robustness checks
- Manual annotation does not scale
- Hard to separate model from harness

Sources: Yehudai et al., "Survey on Evaluation of LLM-based Agents" (2025); Sierra τ-bench; OpenAI (2026)

What changes when we test agents

	LLM application (this talk)	Agent
Test input	One or several utterances	Environment + goal + simulated user
Execution	One response, or a dialogue	Long trajectory of tool calls
Oracle	Judge the response text	Check final state and the trajectory
Fitness	Judge score	Distance to policy violation, cost, wasted steps
Failure	A bad answer	An irreversible action
Reproducibility	Stochastic model	Stochastic model, tools, environment, harness

A path to testing agents

1 · Parametrise the world

Environment features and user personas, as with scene features in ISU-Test

2 · Guide the search

Adversarial users and injected tool failures, like our multi-turn strategies

3 · Oracles on trajectories

Deterministic state checks first; validated LLM judges for the rest

4 · Diversify and repeat

Cluster trajectories, repeat runs (pass^k), report for model + harness

Emerging work (2026 arXiv, titles only): FLARE, coverage-guided fuzzing for multi-agent systems; metamorphic testing with trace-based oracles; LogicHunter, agentic oracle for agent frameworks.

Open challenges

Test adequacy

What is coverage for natural language? Features, embedding clusters and strategies are proxies, and clustering choices change the answer.

Oracles

Judge bias, subjective criteria, cost, and domain terminology the base LLM may not know.

Reproducibility

System, generator and judge are stochastic; API latency varies, so equal time budgets give unequal test counts.

Validity

Are generated inputs realistic, and do the failures we find matter to users?

Takeaways

1. **Benchmarks tell you how good a model is. Testing tells you where your system breaks.**

2. Treat it as search: features, fitness, oracle, and diversity as an explicit goal.

3. Test the oracle too, and report results in a way others can reproduce.

4. The same principles extend to agents: environment + goal, trajectories, and the harness.

Benchmarking Vision-Language Models for In-Car Scene Understanding

ICSE 2027

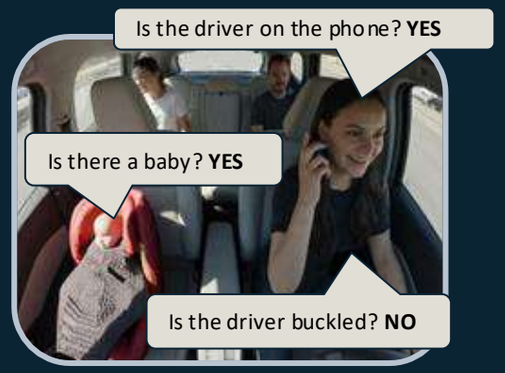
TRACK A
Test Generation



Rendered Scene Modified Scene

- Generate realistic scenes
- Preserve geometry & semantics
- Evaluate failure rate & realism

TRACK B
Robust Perception



- Build your robust ISU-System
- Answer safety questions
- Evaluate accuracy & latency

ISU-Challenge 2027

🤗 Dataset on Hugging Face 🔗 https://isu-challenge.github.io k Starterkit on Kaggle

PARTICIPATE

- ➔ Submit your solution by: **27th November 2026**
- ➔ Secure 1 of the 3 winning places per track
- ➔ Publish your solution paper at ICSE 2027

FAU fortiss

Repo: https://github.com/deeptest-competition

DEEPTEST COMPETITION 2026







REGISTER FOR
ISU-CHALLENGE 27‘!





Sorokin, L., Vasilev, I., & Pasini, S. (2026). *DeepTest Tool Competition 2026: Benchmarking an LLM-Based Automotive Assistant*. DeepTest Workshop. ICSE 2026.

Beyond Benchmarks:

Systematic Testing of LLM-Based Software Systems



Prof. Dr. Andrea Stocco
Chair of Software Engineering for Data-intensive Applications

School of Computation, Information and Technology
Technical University of Munich

Lead of the Automated Software Testing (AST)

Website/X: https://tsigalko18.github.io / @tsigalko18
Email: stocco@fortiss.org / andrea.stocco@tum.de

References

- Sorokin, Vasilev, Friedl, Stocco: STELLAR: A Search-Based Testing Framework for LLM Applications. SANER 2026
- Sorokin, Vasilev, Friedl, Stocco: Diversity-Guided Search-Based Testing of LLM Applications. Empirical Software Engineering
- Negi, Sorokin, Tayebi Arasteh, Stocco: Automated Evaluation of Multi-Turn Dialogues in In-Car Conversational Assistants. ITSC 2026
- Habicht et al.: Benchmarking Contextual Understanding for In-Car Conversational Systems. arXiv 2512.12042
- Giebisch, Friedl, Sorokin, Stocco: Automated Factual Benchmarking for In-Car Conversational Systems. IV 2025
- Sorokin, Yang, Friedl, Stocco: Search-based Testing of Vision Language Models for In-Car Scene Understanding (ISU-Test). ASE 2026
- OpenAI: Why SWE-bench Verified no longer measures frontier coding capabilities (23 Feb 2026)
- Gema et al.: Are We Done with MMLU? arXiv 2406.04127
- Chen et al.: Benchmarking LLMs under Data Contamination, a Survey from Static to Dynamic Evaluation. EMNLP 2025
- Sierra: τ-bench, benchmarking AI agents for the real world (pass^k)
- Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows. arXiv 2605.27922
- Yehudai et al.: Survey on Evaluation of LLM-based Agents. arXiv 2503.16416